

Lista de implementação de Herança e Composição. Implemente na linguagem Java todos os exercícios propostos.

Formato de Entrega

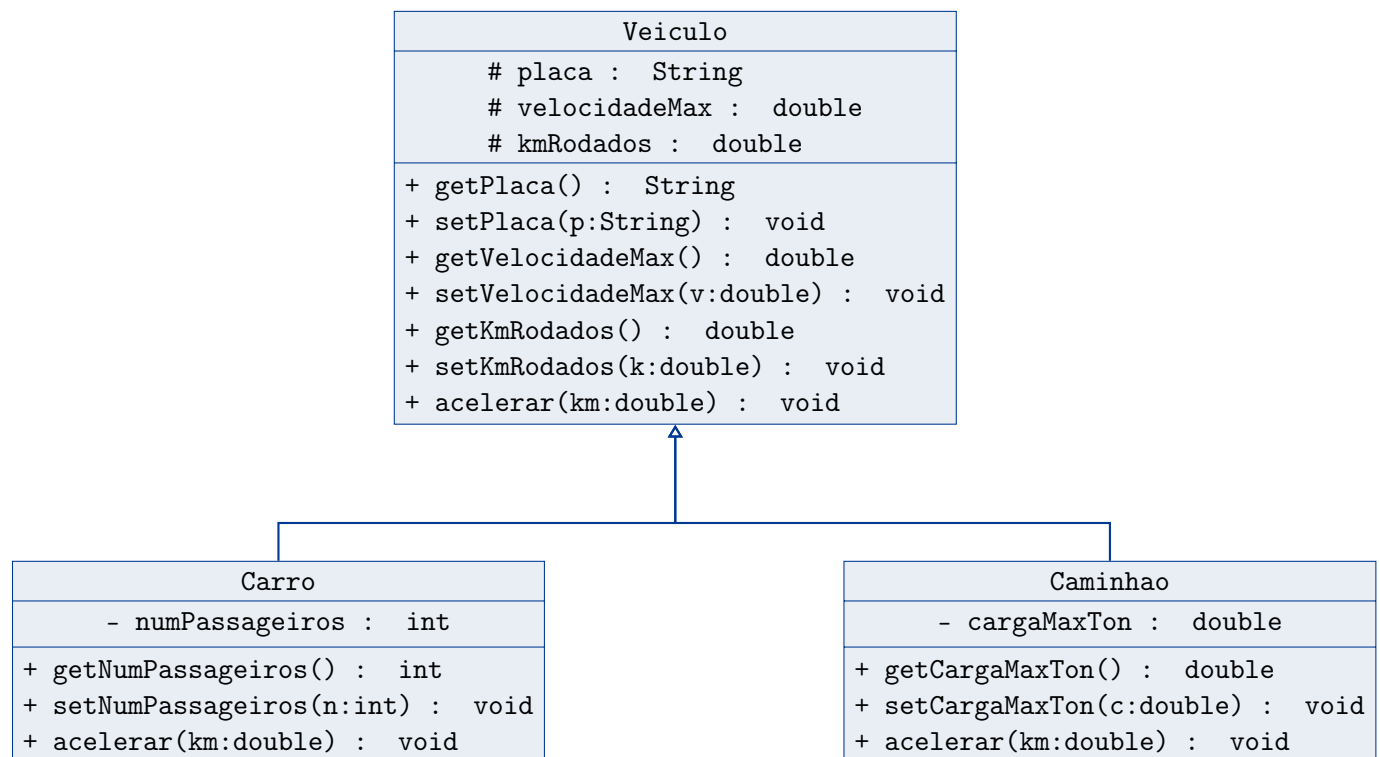
Instruções para Submissão

A entrega desta lista de exercícios deve ser feita através de um **único arquivo .zip**. Dentro deste arquivo, deve haver uma pasta estruturada para cada exercício (ex: Exercicio1, Exercicio2, ..., Exercicio8).

Atenção: Certifique-se de que cada pasta contenha **todos** os arquivos necessários (.java) para a correta compilação e execução do respectivo exercício.

1) Herança — Veículos de Transporte

Implemente uma hierarquia de classes para representar veículos de transporte. A superclasse Veiculo possui atributos e métodos comuns, e as subclasses especializam o comportamento.



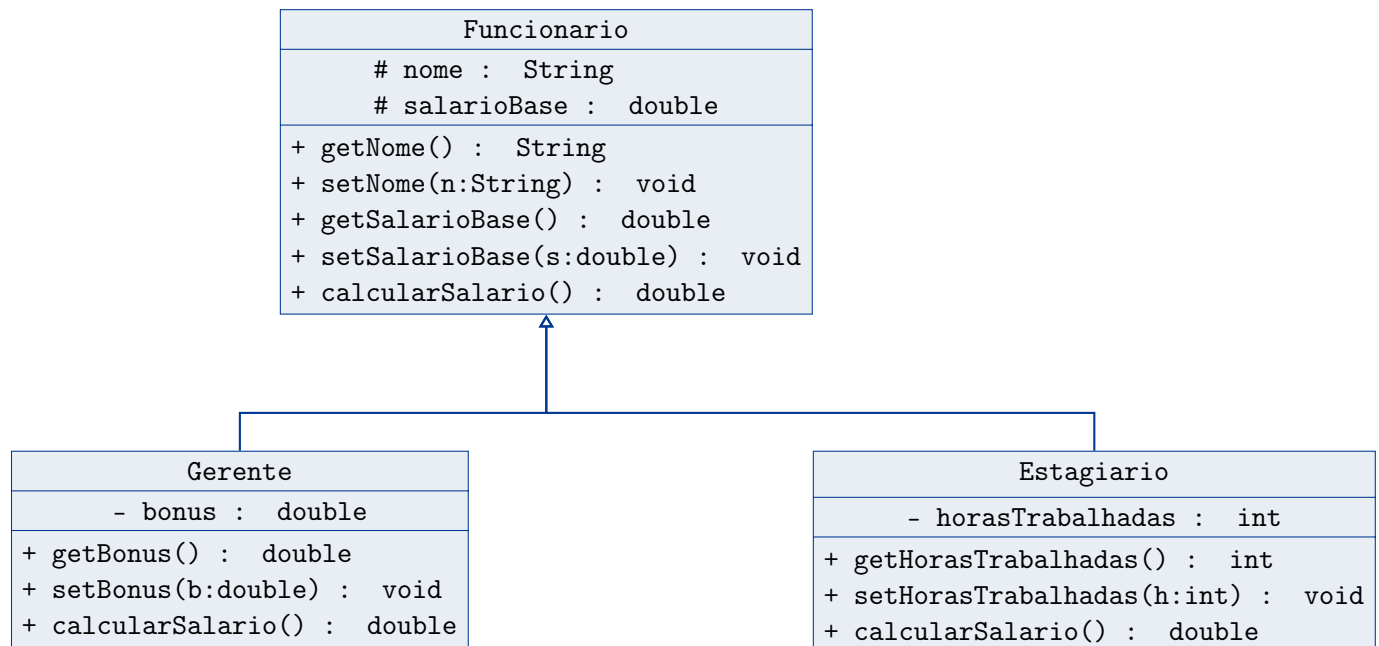
Requisitos

- A classe **Veiculo** deve possuir o método `acelerar(km)` que incrementa `kmRodados`.

- Carro sobreescreve `acelerar()` usando `super` e adiciona uma verificação de limite de velocidade.
- Caminhao sobreescreve `acelerar()` reduzindo a eficiência em 30% se a carga estiver acima de 80% da capacidade.
- Crie uma classe `Main` que instancie ambos os veículos e teste os métodos.

2) Herança — Funcionários de uma Empresa

Modele um sistema de folha de pagamento com uma superclasse `Funcionario` e duas subclasses que calculam o salário de formas diferentes.

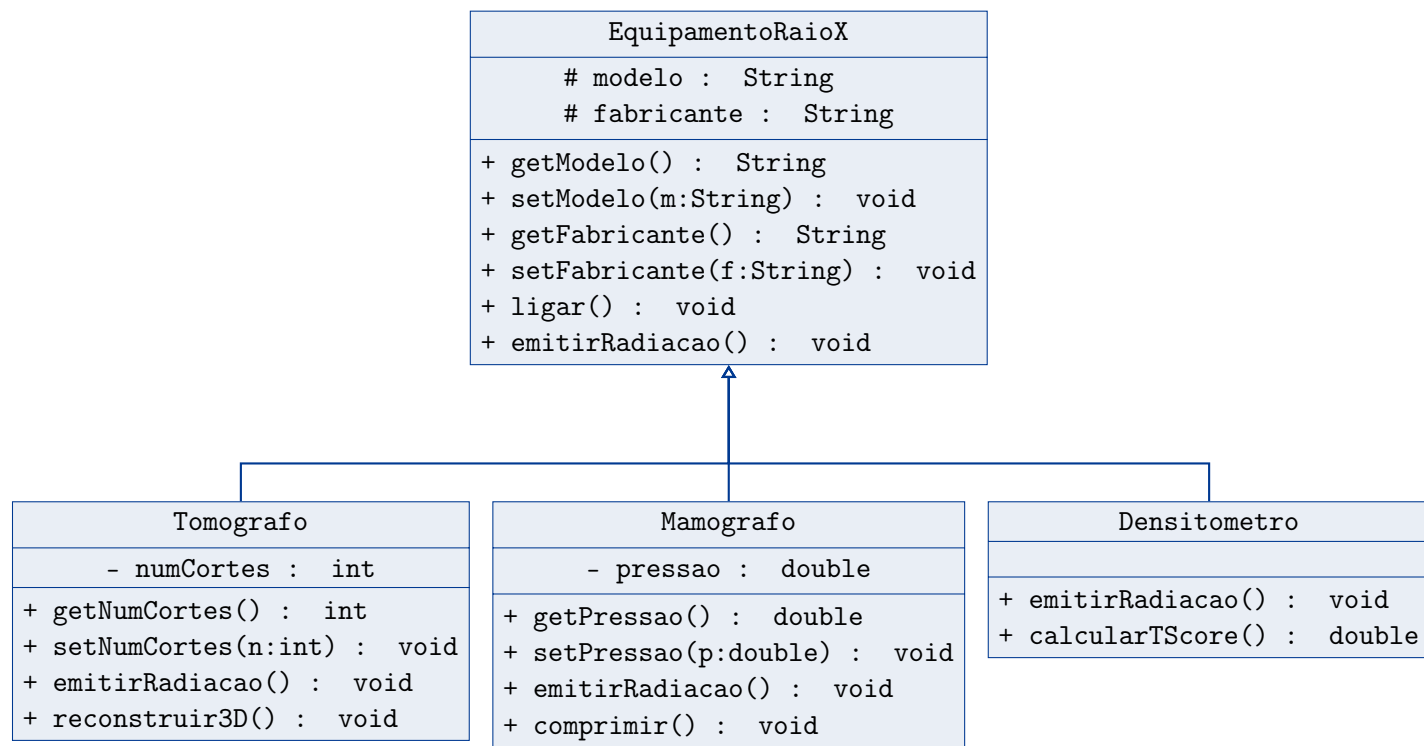


Requisitos

- **Gerente** calcula o salário como `salarioBase + bonus`.
- **Estagiario** calcula o salário com base nas horas trabalhadas multiplicadas por um valor fixo por hora (ex: R\$ 15,00).
- Utilize `super()` para inicializar os atributos da superclasse nos construtores.
- Crie um `Main` que imprima o salário de cada tipo de funcionário.

3) Herança — Equipamentos Médicos

Modele uma hierarquia de classes de equipamentos médicos de raio-X. A superclasse define o comportamento básico de emissão de radiação.

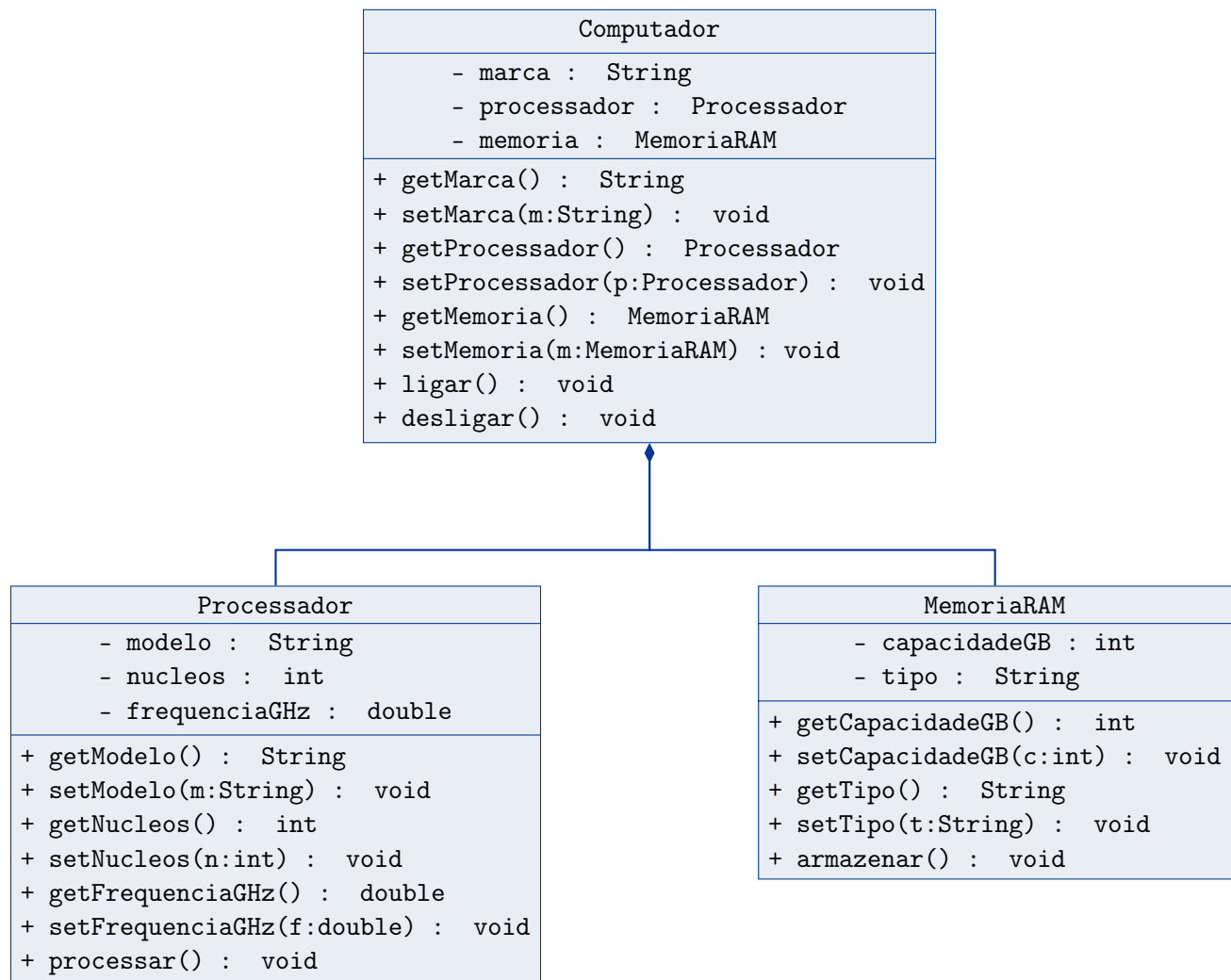


Requisitos

- Cada subclasse deve sobrescrever `emitirRadiacao()` chamando `super.emitirRadiacao()` e adicionando comportamento específico.
- Tomografo imprime a quantidade de cortes realizados.
- Mamografo imprime a pressão de compressão aplicada.
- Densitometro calcula e retorna um T-Score fictício.
- Teste todos os equipamentos em um Main.

4) Composição — Computador e seus Componentes

Implemente um sistema onde um **Computador** é composto por um **Processador** e uma **MemoriaRAM**. Sem o computador, as partes não existem.

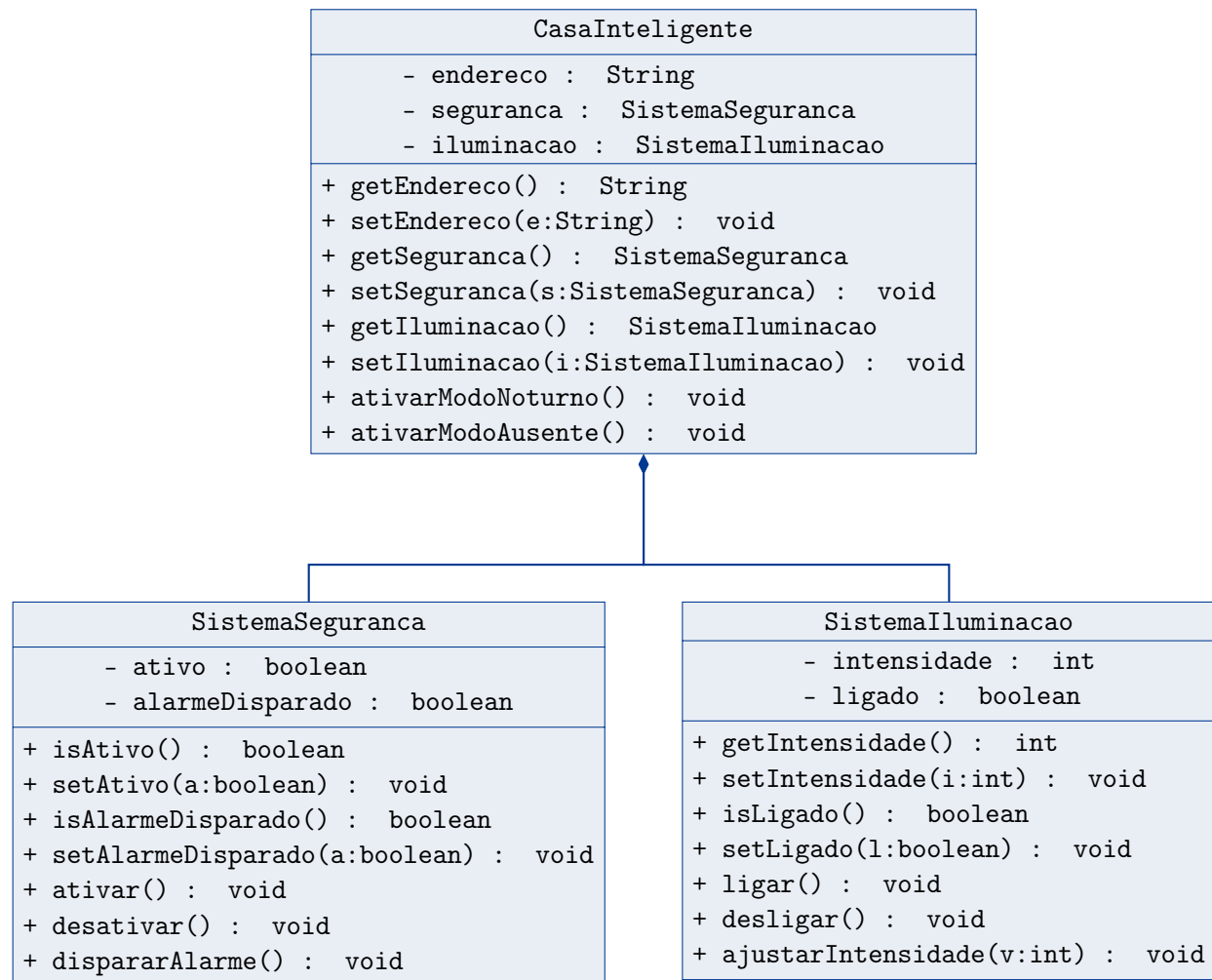


Requisitos

- O Computador deve criar os objetos Processador e MemoriaRAM internamente (composição forte).
- O método ligar() deve delegar para processador.processar() e memoria.armazenar().
- Todos os atributos devem ser privados com acesso via métodos.
- Crie um Main demonstrando a criação e o uso do computador.

5) Composição — Casa Inteligente

Modele uma CasaInteligente composta por um SistemaSeguranca e um SistemaIluminacao. A casa gerencia os subsistemas.

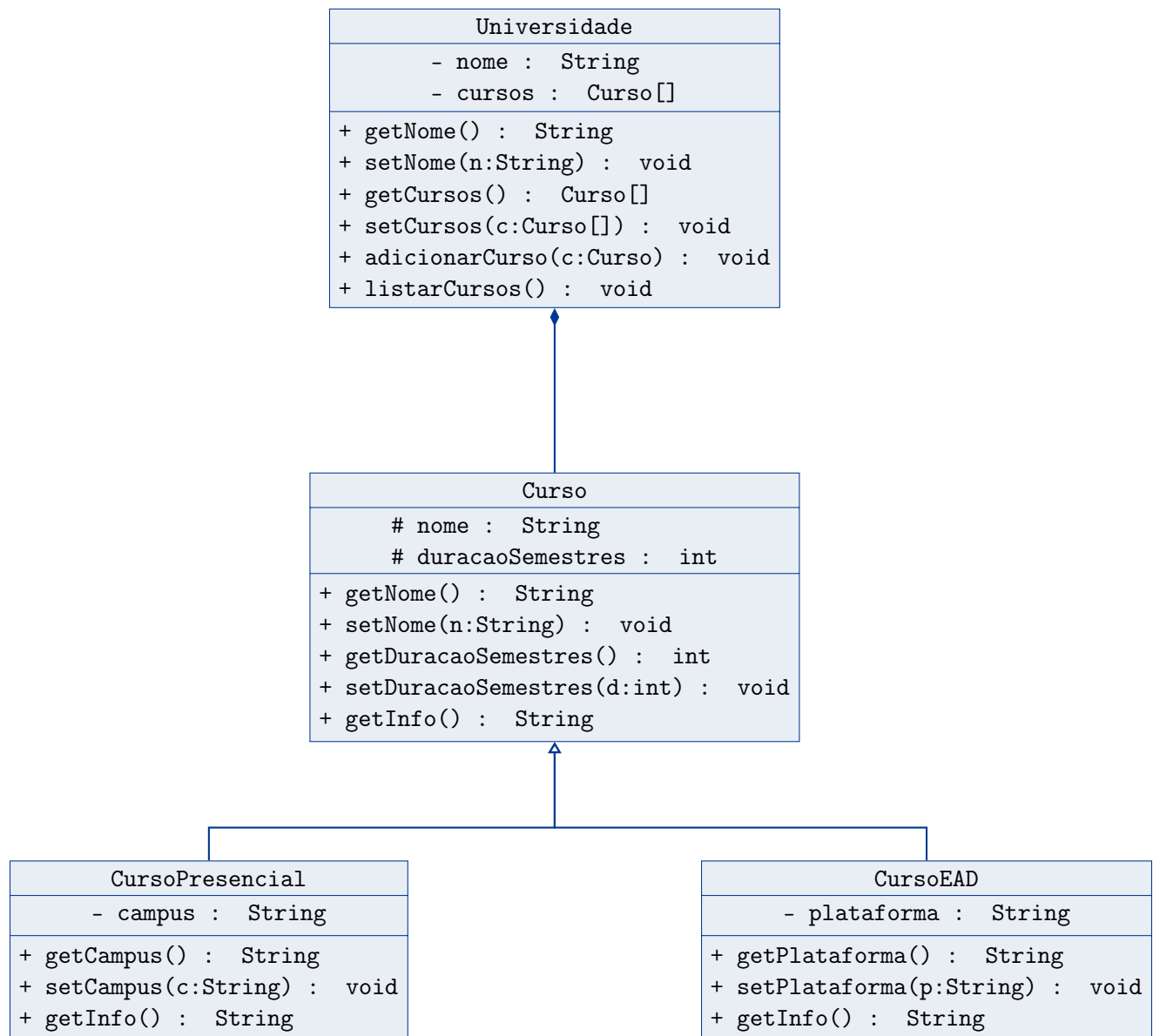


Requisitos

- `ativarModoNoturno()` deve ligar a segurança e reduzir a iluminação para 20%.
- `ativarModoAusente()` deve ativar a segurança e desligar a iluminação.
- Demonstre a delegação de responsabilidades no `Main`.

6) Herança + Composição — Universidade

Combine herança e composição. Uma `Universidade` possui (composição) vários `Cursos`. `Curso` é superclasse de `CursoPresencial` e `CursoEAD`.

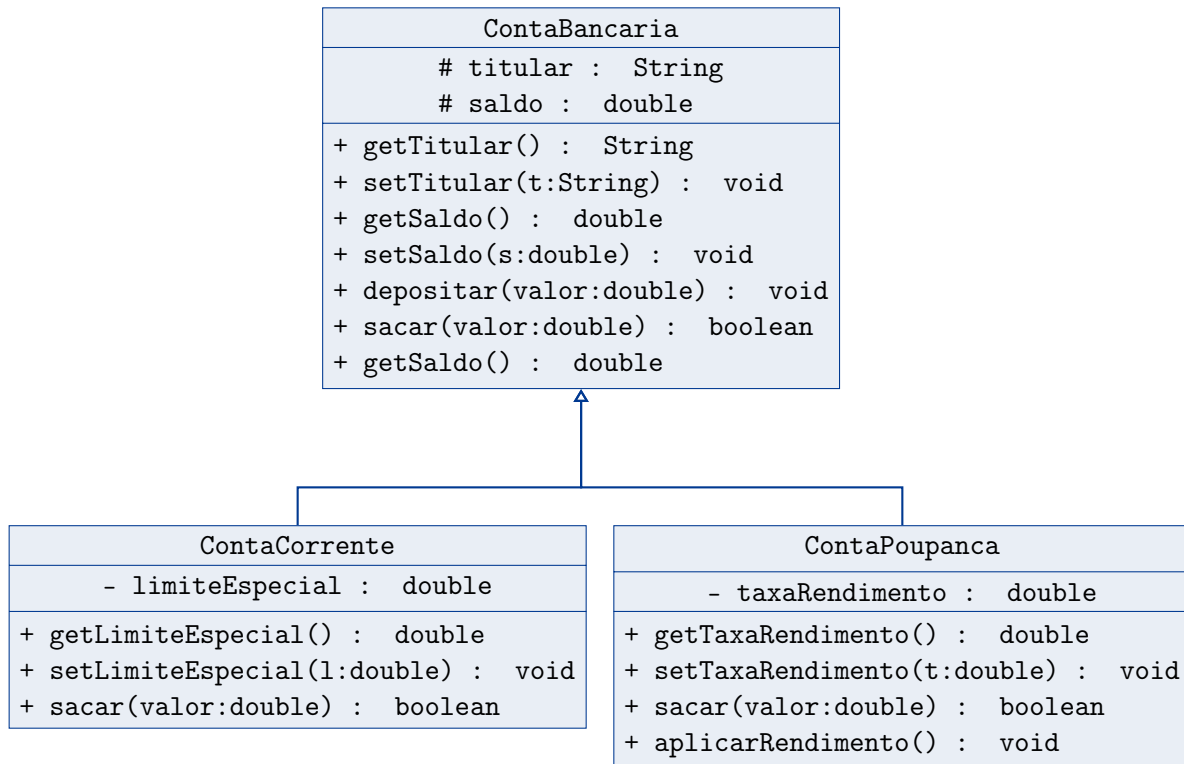


Requisitos

- CursoPresencial inclui o campus no `getInfo()`.
- CursoEAD inclui a plataforma no `getInfo()`.
- A Universidade deve ser capaz de listar todos os cursos com suas informações específicas.
- Utilize `super.getInfo()` para reaproveitar a lógica da superclasse.

7) Herança — Contas Bancárias

Modele um sistema bancário com uma superclasse `ContaBancaria` e duas subclasses com regras de saque distintas.

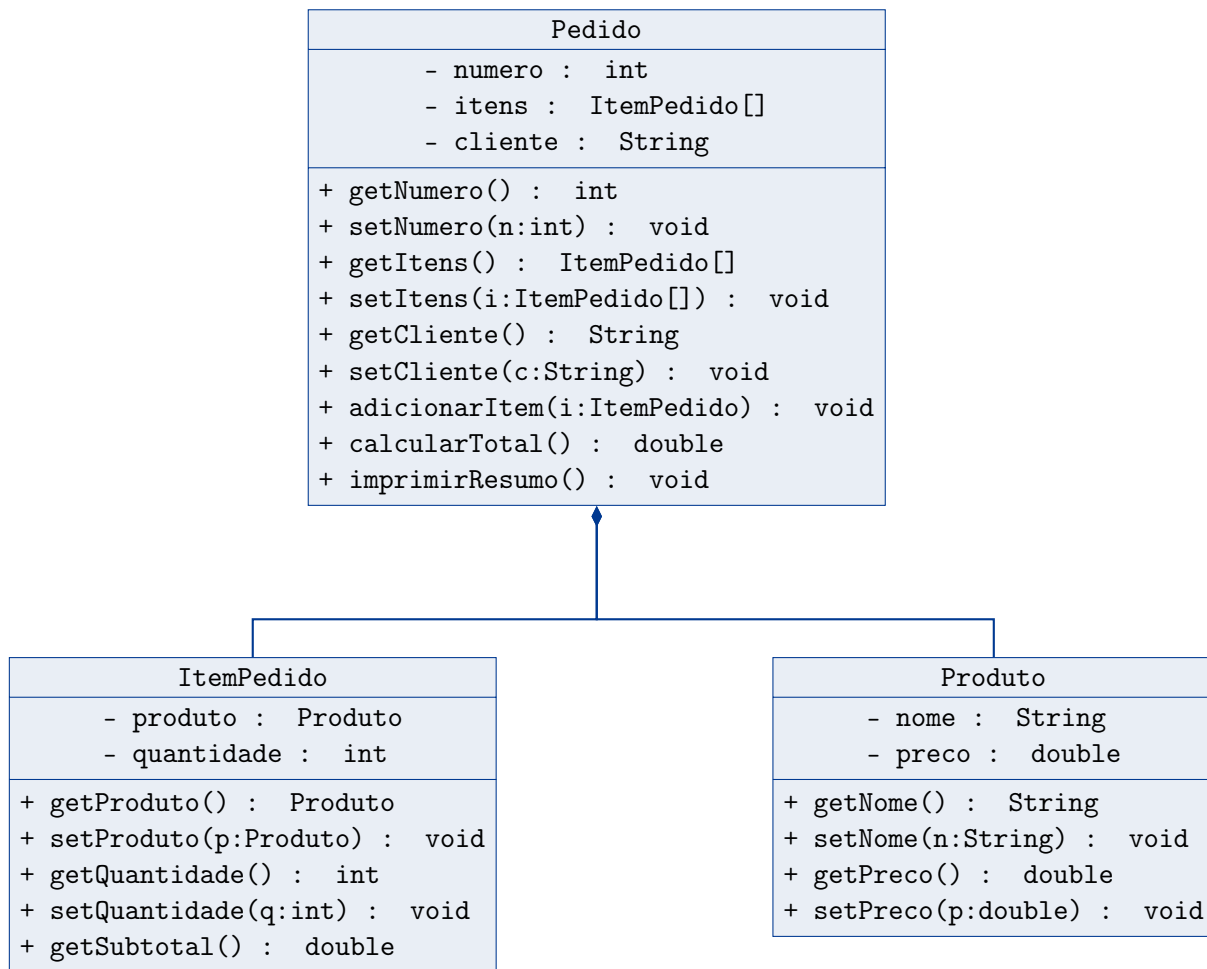


Requisitos

- `ContaCorrente` permite saque até `saldo + limiteEspecial`.
- `ContaPoupanca` só permite saque se o saldo permanecer positivo (sem limite especial).
- O método `aplicarRendimento()` incrementa o saldo com base na `taxaRendimento`.
- Utilize `super()` nos construtores e `super.sacar()` onde fizer sentido.

8) Composição — Pedido de Restaurante

Modele um sistema de pedidos onde um `Pedido` é composto por vários `ItemPedidos`. Cada item referencia um `Produto`.



Requisitos

- O Pedido gerencia a lista de itens e calcula o total somando os subtotais.
- ItemPedido calcula o subtotal como `produto.getPreco() * quantidade`.
- `imprimirResumo()` deve listar todos os itens com nome, quantidade, subtotal e o total geral.
- Crie um Main que monte um pedido com pelo menos 3 itens e imprima o resumo.